Lesson Plans: Weeks 1-3

Embedded Systems Curriculum · 26 Oct - 15 Nov 2026 · @Sam

How to use these plans

- Each week has a goal, a time budget, six sessions, labs with expected results, a cold question with an answer key, and a done-when checklist.
- Rhythm for every lab: predict, build, measure, reconcile. Write your predicted number before you touch the meter. The gap between prediction and measurement is where the learning is.
- Expected values and answer keys are for checking, not reading first. Cover them until you have made an attempt.
- Weekly split: 7 h learning, 8 h building, 3 h documenting, 2 h spare. Session lengths are guides; spare hours float to wherever something broke.
- You already code, so the C thread skips basics and concentrates on what is different in C: fixed-width types, no bounds checks, everything manual.
- Keep one folder per week on your Mac (embedded/week-01 and so on) for notes, photos and sketches. Git starts in week 4.
- All quantities are SI/metric.

Kit for weeks 1-3

- Digital multimeter with a mA current range, 830-point breadboard, jumper wires, small cutters/strippers
- Resistors: 220 Ω, 470 Ω, 1 kΩ, 2.2 kΩ and 10 kΩ (several of each)
- Red LEDs (a handful, plus a few other colours), 9 V battery with snap connector
- Arduino Uno-class board and USB cable (with a USB-C adapter if your Mac needs one). If your board is not an Uno, pin numbers and PWM pins below will differ.
- 2 pushbuttons, a 10 kΩ NTC thermistor (B about 3950), an LDR (photoresistor)
- A notebook for predictions, measurements and schematics

Week 1 (26 Oct): Electricity basics

Goal: By Sunday you can predict, build and measure series, parallel and LED circuits with a multimeter, and redraw them from memory.

Covers: principles of electric circuits, electronics fundamentals, multimeter, breadboarding, basic math.

Sessions (20 h)

- S1, 3.5 h: Orientation. Print the roadmap, highlight the Required items you can already do, and tick them in section 11 (be strict: a tick means no tutorial open). Start the job-posting log (role, link, skills named, certificates, pay, firmware-heavy or field-heavy) and save and tag your first 3 postings (1.25 h). Learn Ohm's law and power (1.25 h). Math hour (1 h).
- S2, 3.5 h: Learn the multimeter, resistor colour codes and breadboard internals (1.25 h). Lab 1.1 (2 h). Notes (0.25 h).
- S3, 3.25 h: Learn series and parallel circuits (1.5 h). Lab 1.2 (1.5 h). Notes (0.25 h).
- S4, 2.75 h: Lab 1.3 (1.5 h). Learn LEDs and diodes (1 h). Notes (0.25 h).
- S5, 3.25 h: Lab 1.4 (2 h). Learn to read schematics (1 h). Notes (0.25 h).
- S6, 3.75 h: Lab 1.5 (1 h). Final notes, cold question, journal (0.75 h). Spare (2 h).

Key ideas

- Ohm's law: $V = IR$. Power: $P = VI = I^2R = V^2/R$. Know all the rearrangements.
- Series: same current, voltages add, resistances add. Parallel: same voltage, currents add, and for two resistors $R = R1R2 / (R1 + R2)$.
- Voltage is measured across a part, current through it. A meter in current mode connected straight across a battery is a short circuit.
- Four-band colour code: two digits, then a multiplier; gold is 5%. 220 Ω red-red-brown. 470 Ω yellow-violet-brown. 1 kΩ brown-black-red. 2.2 kΩ red-red-red. 10 kΩ brown-black-orange.
- Breadboard: each group of 5 holes in a column is joined, the centre trench separates the two halves, and power rails run the length of the board but on some boards break in the middle.
- An LED passes current one way (long leg is the anode) with a roughly fixed forward voltage (about 2 V for red), so a series resistor sets its current. Typical target: 10-20 mA.
- A fresh 9 V battery reads about 9.3-9.6 V unloaded. Use your measured value in predictions.

Lab 1.1: Multimeter first contact (2 h)

1. Measure the battery in DC volts.

2. Measure three resistors; compare with the colour code and the 5% tolerance. Touch only the probe tips, not the resistor leads.
3. Continuity: test a jumper wire, a breadboard column, and whether your rails break at the middle.
4. Diode test an LED with the red probe on the long leg. A red LED shows about 1.8-2.1 V and may glow faintly; reversed, the meter shows OL.
5. Practise moving the red lead to the mA jack and back. Leaving it in the wrong jack is the usual way a meter fuse blows.

Lab 1.2: Series (1.5 h)

1 kΩ and 2.2 kΩ in series across 9 V. Predict total resistance, current and each voltage, then measure the voltages and the current (break the circuit and put the meter in series).

Expected at 9.0 V: 3.2 kΩ total, about 2.8 mA, about 2.8 V across the 1 kΩ and 6.2 V across the 2.2 kΩ. The two voltages should sum to the battery voltage. Swap the two resistors: what changes?

Lab 1.3: Parallel (1.5 h)

The same two resistors in parallel across 9 V. Expected: about 0.69 kΩ combined, branch currents about 9.0 mA and 4.1 mA, total about 13.1 mA. Measure each branch and the total; the branches should add up to the total. Then predict, and test, what a third 1 kΩ in parallel does to the total current.

Lab 1.4: LED on 9 V (2 h)

1. Calculate: R = (9 - 2.0) / 0.015 ≈ 467 Ω, so use the nearest standard value, 470 Ω. Expected current about 15 mA. The resistor dissipates about 0.1 W, inside the 0.25 W rating of a quarter-watt resistor.
2. Build it. Measure the voltage across the LED, across the resistor, and the current. The two voltages should sum to the supply.
3. Swap in 1 kΩ. Expected about 7 mA. It is dimmer, but not half as bright to your eye.
4. Never run an LED without a resistor, and do not go below 470 Ω on 9 V.

Lab 1.5: Schematics (1 h)

The day after building, draw each circuit from memory: battery, resistor, LED, ground, and a wire junction versus a crossing. Label component values and the node voltages you measured. Then rebuild one circuit on the breadboard from your drawing alone.

Math hour: algebra, units and SI prefixes

1. Convert 4.7 kΩ to Ω, 2200 Ω to kΩ, 15 mA to A, and 250 µA to mA.
2. 5 V across 220 Ω: what current?
3. 20 mA through 330 Ω: what voltage?
4. 3.3 V and 10 mA: what resistance?
5. Power for 9 V at 15 mA?
6. A 470 Ω resistor rated 250 mW: maximum continuous current? (Rearrange $P = I^2R$.)
7. Rearrange $V = IR$ and $P = VI$ to give P in terms of V and R only.

Answers: (1) 4700 Ω, 2.2 kΩ, 0.015 A, 0.25 mA. (2) 22.7 mA. (3) 6.6 V. (4) 330 Ω. (5) 135 mW. (6) $\sqrt{(0.25 / 470)} \approx 23$ mA. (7) $P = V^2/R$.

Cold question

What resistor does a red LED need on 9 V, and why?

Answer key: about 470 Ω. The LED drops roughly 2 V and wants about 15 mA, so the resistor must drop the other 7 V: 7 / 0.015 ≈ 467 Ω, rounded to the standard value. Without it the LED's current is limited only by the battery, and the LED fails.

Done when

- ☐ Solve Ohm's law for voltage, current or resistance without notes
- ☐ Build and measure a series circuit and a parallel circuit with a multimeter: volts, amps in series, ohms, continuity, diode test
- ☐ Explain breadboard internals; read resistor colour codes
- ☐ Calculate, build and measure an LED circuit on a 9 V battery
- ☐ Draw schematics of circuits you built, from memory

File this week

Notebook pages (predictions, measurements, errors), three schematics, a photo of each build, your skills-map ticks, the first 3 tagged postings, and the Sunday journal entry.

Week 2 (2 Nov): Arduino basics

Goal: By Sunday, from a blank sketch and without a tutorial, you can blink one LED without delay(), toggle another with a debounced button, fade a third with PWM, and print events to the Serial Monitor.

Covers: GPIO, PWM, state machines (start), C (start).

Needs: Arduino Uno-class board, USB cable, breadboard, 3 LEDs, 3 × 220 Ω, a pushbutton, jumper wires.

Sessions (20 h)

- S1, 3 h: C thread hour 1 (1 h). Install Arduino IDE 2.x, pick the board and port, and learn the Arduino model (1 h). Lab 2.1 (1 h).
- S2, 3.5 h: Learn digital input, pull-ups and debounce (1 h). Lab 2.2 (2 h). Notes (0.5 h).
- S3, 4 h: C thread hour 2 (1 h). Learn why delay() hurts and how millis() fixes it (1 h). Lab 2.3 part 1 (1.5 h). Notes (0.5 h).
- S4, 3.5 h: Lab 2.3 part 2 (1 h). Learn PWM (1 h). Lab 2.4 part 1 (1 h). Notes (0.5 h).
- S5, 3.5 h: Math hour (1 h). Lab 2.4 part 2 and Lab 2.5 (1.5 h). Tag 3 more postings (0.5 h). Notes (0.5 h).
- S6, 2.5 h: Cold question and journal (0.5 h). Spare (2 h): use it for the self-test below.

Key ideas

- setup() runs once, loop() repeats forever. The core calls are pinMode(), digitalWrite() and digitalRead().
- Uno pins: digital 0-13, analog A0-A5. Pins 0 and 1 are shared with the USB serial link, so leave them alone. PWM-capable pins are marked ~ : 3, 5, 6, 9, 10, 11. Pin 13 drives the on-board LED.
- Each pin: 20 mA recommended, 40 mA absolute maximum. A red LED with 220 Ω on 5 V draws about (5 - 2) / 220 ≈ 14 mA.
- A floating input reads noise. INPUT_PULLUP turns on an internal pull-up (roughly 20-50 kΩ), so wire the button between the pin and GND: the pin reads HIGH at rest and LOW when pressed.
- Contacts bounce for a few milliseconds. Ignore changes until the reading has been stable for about 20 ms, and toggle on the press edge (HIGH to LOW), not on the level.
- delay() halts the whole sketch, so button presses during it are missed. millis() returns milliseconds since power-up as an unsigned long (rolls over after about 49.7 days); compare with subtraction and it keeps working across the rollover.
- analogWrite(pin, 0-255) switches the pin on and off quickly; duty cycle is value / 255. Pins 5 and 6 run at about 980 Hz, the others at about 490 Hz. digitalWrite gives only 0 V or 5 V; analogWrite gives a pulse train whose average lies between.
- A state variable such as ledState is your first state machine: the sketch remembers what it is doing between passes of loop(). Week 6 builds on this.

Patterns to recognise (attempt your own first):

```
// Non-blocking timer
unsigned long previous = 0;
const unsigned long INTERVAL = 500;   // ms
...
if (millis() - previous >= INTERVAL) {
  previous += INTERVAL;
  // do the job
}
```

```
// Debounced press detection
bool lastReading = HIGH, stable = HIGH;
unsigned long lastChange = 0;
...
bool reading = digitalRead(BUTTON_PIN);
if (reading != lastReading) { lastChange = millis(); lastReading = reading; }
if (millis() - lastChange > 20 && reading != stable) {
  stable = reading;
  if (stable == LOW) ledState = !ledState;   // pressed
}
```

Lab 2.1: Blink (1 h)

Upload the Blink example to the on-board LED. Change the timing to 200 ms and 1 s. Then move to an external LED on pin 8 (pin, 220 Ω, LED anode, LED cathode, GND). Predict the current, then check it with the meter in series.

Lab 2.2: Button toggle (2 h)

Button between pin 2 and GND, LED on pin 8. First write the naive version (toggle whenever the pin reads LOW) and watch it misbehave. Then fix it with press-edge detection, then add debounce. Requirement: one press is one toggle, and holding the button does not retrigger.

Lab 2.3: Two jobs at once (2.5 h)

LED A on pin 7 blinks every 500 ms using millis(), while the button from Lab 2.2 toggles LED B, and neither feels laggy. Then add an uptime print every 1000 ms. To prove the point, swap in delay(500) and count the presses you miss.

Lab 2.4: PWM fade (1.5 h)

LED on pin 9. Fade 0 to 255 and back, advancing one step every 10 ms using millis(). Then put the LED on pin 8 and call analogWrite(8, 128): predict, observe, and explain (a non-PWM pin just goes HIGH at 128 or above and LOW below). Stretch: your eye sees

brightness non-linearly, so try writing value * value / 255 for a smoother-looking fade.

Lab 2.5: Serial Monitor (1 h)

Serial.begin(9600) and set the Monitor to the same baud rate. Print only on events (press, toggle, timer tick) with a millis() timestamp. Open the Serial Plotter on the fade value and look at the triangle wave. Printing on every pass of loop() floods the monitor and slows the sketch.

Self-test (S6 spare time): from a blank sketch, in 30 minutes with no tutorial, blink A with millis(), toggle B with a debounced button, fade C, and print events. That is this week's tick.

C thread (2 h)

- Hour 1: install the command-line tools with xcode-select --install. Compile and run a hello world from the Terminal:

```
#include <stdio.h>
int main(void) {
  puts("hello, embedded");
  return 0;
}
```

Build it with: cc -Wall -Wextra hello.c -o hello && ./hello. Then variables, types (int, float, char, const), loops and expressions.

- Hour 2: functions and prototypes. Write float duty_to_volts(int value) returning 5.0f * value / 255, plus a main() with a for loop that prints duty percent and average volts for 0, 64, 128, 192 and 255. Fix every warning.
- Different from languages you know: types are fixed at compile time, integer division truncates, uninitialised locals hold garbage, nothing checks array bounds, and compiler warnings are your first debugger.

Math hour: functions, graphs, duty cycle, y = mx + b

1. The period of a 490 Hz and a 980 Hz signal?
2. Duty cycle and average voltage for analogWrite(64) on 5 V?
3. Which analogWrite value gives about 3.0 V average?
4. Write average voltage as a line y = mx + b of the analogWrite value. What are m and b?
5. A sensor reads 200 at 0 °C and 800 at 100 °C. Write reading = mT + b, then invert it. What temperature is a reading of 440?

Answers: (1) 2.04 ms and 1.02 ms. (2) 25%, 1.25 V. (3) 153. (4) V = 0.0196 × value, so m ≈ 0.0196 V per step and b = 0. (5) reading = 6T + 200, T = (reading - 200) / 6, so 40 °C. This is the shape of the calibration you will do in week 18.

Cold question

How do digitalWrite() and analogWrite() differ, and why only some pins? Why is delay() a problem when a device must do two jobs?

Answer key: digitalWrite sets a pin fully HIGH (5 V) or LOW (0 V). analogWrite outputs PWM, a fast on/off cycle with duty value / 255, so the average voltage lies in between; only pins wired to a hardware timer output (3, 5, 6, 9, 10, 11 on the Uno) can do it. delay() blocks the processor, so every other job waits and events during the delay are missed. millis() lets each pass of loop() check what is due.

Done when

- ☐ Install the Arduino IDE; upload Blink; change the timing
- ☐ A button toggles an LED using a state variable
- ☐ Blink without delay() using millis(), so the board can do two things at once
- ☐ Fade an LED with PWM on a PWM pin
- ☐ Print values with the Serial Monitor
- ☐ C: compile and run hello world from the Terminal; variables, types, loops, functions

File this week

Sketches with a comment on every state variable and timing constant, a wiring photo with a pin table, notes on what bounce looked like before you fixed it, 3 more tagged postings, and the journal entry.

Week 3 (9 Nov): Sensors

Goal: Read a thermistor and an LDR in the same loop, convert to °C, explain every number in the pipeline, and print CSV lines ready for Python in week 4.

Covers: ADC/DAC, electronics fundamentals, digital design, sensors, C.

Needs: Arduino board, breadboard, 10 kΩ NTC thermistor (B about 3950), LDR, 10 kΩ resistors, a 2.2 kΩ resistor, jumper wires, multimeter.

Sessions (20 h)

- S1, 3 h: Learn ADC, DAC and voltage dividers (1 h). Lab 3.1 (1.5 h). Notes (0.5 h).

- S2, 4 h: Learn thermistors and the Beta equation (1 h). Math hour (1 h). Lab 3.2 part A (1.5 h). Start the datasheet cards (0.5 h).
- S3, 3.5 h: C thread hour 1 (1 h). Digital design hour 1 (1 h). Lab 3.2 part B (1 h). Datasheet cards (0.5 h).
- S4, 3.5 h: Digital design hour 2 (1 h). Lab 3.3 (2 h). Tag 3 more postings (0.5 h).
- S5, 3.5 h: C thread hour 2 (1 h). Lab 3.4 (1 h). Lab 3.5 (1 h). Pin table, photos, notes (0.5 h).
- S6, 2.5 h: Cold question and journal (0.5 h). Spare (2 h).

Key ideas

- The Uno's ADC is 10-bit: 0-1023 counts across 0 V to the reference (5 V by default), so one count is about 4.9 mV. Voltage ≈ count × 5.0 / 1023. Dividing by 1024 is also common; the difference is under 0.1%.
- A DAC does the reverse: it turns a number into a voltage. The Uno has none; PWM plus a filter approximates one.
- Voltage divider: Vout = Vin × R2 / (R1 + R2). The ATmega328P wants a source resistance of about 10 kΩ or less feeding the ADC, which is why 10 kΩ parts are common.
- The measurement is ratiometric: the divider and the ADC both reference the same 5 V rail, so a sagging USB rail mostly cancels out of the reading.
- An NTC thermistor's resistance falls as it warms, nonlinearly. Beta model: 1/T = 1/T0 + ln(R/R0) / B, with T in kelvin, T0 = 298.15 K (25 °C), R0 = 10 kΩ, and B from the datasheet. In Arduino C, log() is the natural log.
- Wiring used here: 5 V, 10 kΩ fixed resistor, A0, thermistor, GND. Then R_therm = R_fixed × count / (1023 - count), and counts fall as it warms. Swap the two parts and the formula flips, so write down which way you wired it.
- Resolution is not accuracy: near 25 °C one count is about 0.09 °C (about 11 counts per °C), but thermistor tolerance and B uncertainty make real errors of a degree or more.

Lab 3.1: Voltage divider (1.5 h)

1. Build 10 kΩ over 10 kΩ from 5 V to GND. Predict, then measure the midpoint (expect 2.50 V). Then 10 kΩ on top and 2.2 kΩ below (expect 0.90 V).
2. Connect the midpoint to A0 and print analogRead(). Expect about 512 and 184 counts.
3. Convert counts to volts in code and compare with the meter. Note the difference in mV.

Lab 3.2: Thermistor to °C (2.5 h)

- Part A (1.5 h): wire the thermistor divider and print raw counts. Expect about 570 at 20 °C and 512 at 25 °C. Pinch the thermistor and watch counts fall. Calculate R_therm from the counts and check it against the datasheet table.
- Part B (1 h): write adc_to_celsius() as a pure function with no Arduino calls, so you can test it on your Mac later. Average 16 readings. Compare with a reference thermometer; expect agreement within 2-3 °C and list the error sources (part tolerance, B value, self-heating, noise).

Reference version, for checking after your own attempt:

```
const float R_FIXED = 10000.0f;
const float R0 = 10000.0f;
const float T0_K = 298.15f;
const float BETA = 3950.0f;   // check your datasheet

float adc_to_celsius(int adc) {
  if (adc <= 0 || adc >= 1023) return NAN;   // open or shorted sensor
  float r = R_FIXED * adc / (1023.0f - adc);
  float inv_t = 1.0f / T0_K + logf(r / R0) / BETA;
  return 1.0f / inv_t - 273.15f;
}
```

Lab 3.3: LDR and two sensors in one loop (2 h)

Wire 5 V, LDR, A1, 10 kΩ, GND, so brighter light gives a higher count. Note the raw range in the dark, in room light and under a phone torch. An LDR is uncalibrated, so log a raw light index and do not call it lux. Read both sensors every 500 ms with millis() in one loop and print CSV: a header line once in setup (ms,adc_temp,temp_c,adc_light), then one line per sample. If one channel's reading is affected by the other, discard the first conversion after switching channels. Keep this sketch: Python reads it in week 4.

Lab 3.4: Bits in code (1 h)

With uint8_t x = 0xA5, set bit 4, clear bit 2, toggle bit 7 and test bit 0, printing each result with Serial.print(x, BIN) and in HEX. Then build a status byte (bit 0 thermistor fault, bit 1 dark, bit 2 logging on) that your sensor loop sets and clears. Finally pack the bytes 0x12 and 0x34 into a uint16_t and print it; expect 0x1234, which is 4660. You will use that move to read 16-bit I2C sensors in week 8.

Lab 3.5: Probing the ADC (1 h)

Connect A0 to GND, then to 5 V, then to the 3.3 V pin. Expect 0, 1023 and about 675. Leave A0 floating and watch it wander; explain why. Compare the computed 3.3 V reading with the meter, in mV. Time one analogRead() with micros(); expect around 100 µs.

Datasheet cards (1 h)

For your thermistor and for the ADC chapter of the ATmega328P datasheet, write a half-page card in your own words: absolute maximum ratings versus recommended operating conditions, pinout, the key electrical characteristics (thermistor: nominal resistance, B and tolerance, dissipation constant for self-heating; ADC: resolution, absolute accuracy, input range, reference options, recommended source impedance), and the typical application circuit. Read the ratings tables first, every time.

Digital design (2 h)

- Hour 1: write the truth tables for AND, OR, NOT and XOR from memory. Convert between binary, hex and decimal. Bits are numbered from 0 at the right; one hex digit is one nibble.
- Hour 2: masks and shifts. Set: x |= (1 << n). Clear: x &= ~(1 << n). Toggle: x ^= (1 << n). Test: (x >> n) & 1. A left shift by n multiplies by 2 to the n; a right shift divides.
- Practise on 8-bit values: 0xA5 & 0x0F, 0xA5 | 0x0F, 0xA5 ^ 0xFF, 0x3C << 2, 0xA5 >> 4, and 0b10110110 in hex.
- Answers: 0x05, 0xAF, 0x5A, 0xF0, 0x0A, 0xB6.

C thread (2 h)

- Hour 1: fixed-width integers from stdint.h (uint8_t, uint16_t, int32_t). On your Mac, print sizeof for char, short, int, long, long long, float, double and a pointer; expect 1, 2, 4, 8, 8, 4, 8, 8 on Apple silicon. Run the same on the Uno through the Serial Monitor: int is 2 bytes, long 4, double 4, a pointer 2. This is why embedded code uses sized types.
- Hour 2: pitfalls, each predicted before you run it, compiled with -Wall -Wextra:
  - Overflow wraps: a uint8_t holding 255, plus 1, is 0.
  - Promotion: with uint8_t a = 200, b = 100, a uint16_t c = a + b gives 300, but a uint8_t d = a + b gives 44.
  - int is 16-bit on the Uno, so adc * 5000 overflows. Write (uint32_t)adc * 5000UL / 1023 for millivolts; for 512 that gives 2502.
  - Integer division truncates: 5 / 2 is 2; use 5.0f / 2 for 2.5.
  - Precedence trap: if (x & MASK == 0) parses as x & (MASK == 0). Write ((x & MASK) == 0).

Math hour: exponentials and logs

The Beta model says R = R0 × exp(B(1/T - 1/T0)). Taking ln turns that exponential into a straight line in 1/T versus ln(R), which is why the code calls log().

1. Compute ln(R/R0) for R = 5 kΩ, R0 = 10 kΩ.
2. Use it to find T (B = 3950).
3. Predict R at 0 °C (273.15 K), and the ADC count with the 10 kΩ fixed resistor.
4. Predict the counts at 25 °C and 50 °C. Plot the three points and draw a straight line from 0 °C to 50 °C. How far off is the line at 25 °C?

Answers: (1) -0.693. (2) 1/T = 1/298.15 - 0.693/3950, so T ≈ 314.6 K ≈ 41.5 °C. (3) about 33.6 kΩ, about 788 counts. (4) 512 and about 270. The straight line gives about 529 at 25 °C, 17 counts off, which is about 1.5 °C. That is why the conversion needs the ln.

Cold question

If analogRead() returns 512, what voltage is on the pin? What is 0xA5 in binary, and how do you read bit 3?

Answer key: about 2.50 V (512 × 5 / 1023), with one count worth about 4.9 mV. 0xA5 is 1010 0101. Bit 3 is 0; read it with (x >> 3) & 1, or with x & (1 << 3) compared against zero.

Done when

- ☐ Explain ADC: 10 bits, 0-1023, mapping to voltage; explain what a DAC does in a sentence
- ☐ Build and calculate a voltage divider
- ☐ Read a thermistor and convert to °C
- ☐ Read a second sensor (LDR) in the same loop
- ☐ Find pinout, voltage and range in a real datasheet
- ☐ Digital design: truth tables; binary and hex; bit masks and shifts in code
- ☐ C: integer sizes (uint8_t, uint16_t), operators, bit manipulation

File this week

The CSV-printing sketch, your adc_to_celsius() function, the two datasheet cards, divider photos with a pin table, 3 more tagged postings, and the journal entry. Keep the CSV column order stable: week 4 starts by reading it over pyserial.

Kit for weeks 4-6

- Python 3 on your Mac (pyserial and matplotlib in a virtual environment) and a GitHub account
- Soldering iron with temperature control (about 320-350 °C; the lower end for leaded solder), a stand, brass-wool tip cleaner, 0.6-0.8 mm rosin-core solder, desoldering wick and/or a pump, flush cutters, wire strippers
- Safety glasses, and fume extraction or a well-ventilated space
- Plain perfboard with 2.54 mm pitch (check whether yours is isolated pads or stripboard), stranded hookup wire (about 0.2-0.35 mm²), heat-shrink tubing, pin headers, and scrap components to practise on
- Week 6: a makerspace with a 3D printer and an oscilloscope (10x probe), calipers or a ruler, M3 screws and standoffs
- Book your makerspace induction and a print slot during week 5. Waiting for the printer is the slowest part of week 6.

Week 4 (16 Nov): Serial communication and data logging

Goal: By Sunday, a Python script on your Mac logs your week 3 sensor output to a timestamped CSV, you can plot it, and everything is on GitHub with a README.

Covers: UART, Python, Git, C, compilers/GCC, Make, algorithms and data structures (start).

Needs: your week 3 circuit and sketch, USB cable, Python 3, a GitHub account.

Sessions (20 h)

- S1, 3.5 h: Learn UART, I2C and SPI at one-sentence depth (1 h) and the pyserial and CSV approach (1 h). Lab 4.1 (1 h). Notes (0.5 h).
- S2, 3.5 h: Lab 4.2 (2.5 h). Git refresher (0.5 h). Notes (0.5 h).
- S3, 3.5 h: C thread hour 1 (1 h). Learn the compiler stages (1 h). Lab 4.4 part 1 (1 h). Notes (0.5 h).
- S4, 3 h: C thread hour 2 (1 h). Lab 4.4 part 2 (1 h). Learn arrays and structs as data structures (0.5 h). Tag 3 more postings (0.5 h).
- S5, 3.5 h: Math hour (1 h). Lab 4.3 (1.5 h). Lab 4.5 (1 h).
- S6, 3 h: README (0.75 h). Cold question and journal (0.25 h). Spare (2 h).

Key ideas

- UART is asynchronous with no clock: TX crosses to RX, grounds are shared, and both ends agree a baud rate. The usual 8N1 frame is a start bit, 8 data bits (least significant first), no parity and one stop bit, so 10 bits per byte. 9600 baud is about 960 bytes/s; 115200 is about 11,500.
- I2C: two shared wires (SDA, SCL) carry many devices, each with an address, and it needs pull-up resistors. SPI: four wires (clock, MOSI, MISO, chip select), faster, no addresses, one chip-select line per device. Week 8 does both properly.
- The Uno's USB link is a UART: a USB-serial chip talks to pins 0 and 1. On a Mac the port appears as /dev/cu.usbmodem... (genuine boards) or /dev/cu.usbserial... or /dev/cu.wchusbserial... (many clones).
- Only one program can hold a port at a time. Close the Arduino Serial Monitor before you run Python.
- Opening the port resets most Uno boards. Wait about 2 s before reading, and expect one partial or garbage line at the start.
- Timestamp on the receiving side in UTC (ISO 8601), and keep the Arduino millis() column too so you can spot dropped lines.
- A line protocol is a design decision: one sample per line, fixed column order, newline-terminated. Prefix any debug text with # so the reader can ignore it.

Lab 4.1: Arduino side and finding the port (1 h)

Reuse your week 3 sketch at Serial.begin(115200). Make sure it prints nothing except the header and CSV lines (debug lines start with #). Find your port two ways: Tools > Port in the IDE, and in the Terminal by running ls /dev/cu.* before and after plugging the board in.

Lab 4.2: Python reader (2.5 h)

Create a virtual environment and install pyserial. Requirements for the script:

- Takes the port as a command-line argument
- Waits about 2 s after opening, then reads lines, decodes and strips them
- Skips blank, comment and malformed lines without crashing
- Prepends a UTC timestamp and appends to log.csv, writing the header only if the file is new
- Flushes after every row and exits cleanly on Ctrl-C

Shape to aim for:

```
import csv, sys, serial
from datetime import datetime, timezone

port = sys.argv[1]   # e.g. /dev/cu.usbmodem1101
ser = serial.Serial(port, 115200, timeout=1)
# loop:
#   line = ser.readline().decode('utf-8', errors='ignore').strip()
#   skip blank, #-comment and malformed lines
#   row = [datetime.now(timezone.utc).isoformat()] + line.split(',')
#   writer.writerow(row); f.flush()
```

Run it for 10 minutes. Then unplug the USB cable mid-run and see what happens, and make the script report the problem and exit cleanly.

Lab 4.3: Plot (1.5 h)

Log at least 30 minutes with deliberate events (breathe on the thermistor, cover the LDR). With matplotlib draw two stacked plots on a shared time axis, with axis labels and units, a title, and the events marked. Save a PNG for the README. Design it deliberately: it should read clearly at README size.

Lab 4.4: C program with a Makefile (2 h)

Write a C program that reads your log.csv, parses each line into a struct (timestamp skipped with strchr, then sscanf for the numeric fields, using field widths so nothing can overflow), and prints the minimum, maximum and mean temperature. Compute the statistics in functions that take a pointer and a length. Compile with -Wall -Wextra -fsanitize=address,undefined and fix everything it reports.

Then write a Makefile with CC and CFLAGS variables and the targets all, run and clean, marked .PHONY where appropriate. Each recipe line starts with a tab, not spaces. Check that running make twice does nothing the second time, and that touching the source triggers a rebuild.

Lab 4.5: Git and GitHub (1 h)

Structure: README.md, firmware/ (the sketch), logger/ (reader and plot scripts), c/ (the C program and Makefile), docs/ (the PNG), requirements.txt and a .gitignore (the virtual environment, **pycache**, large logs; keep one small sample CSV). Commit in small steps with messages that say why. Push to GitHub, then clone it fresh into a temporary folder and confirm it builds and runs.

C thread (2 h)

- Hour 1: pointers and arrays. Address-of and dereference, pointer arithmetic, how an array becomes a pointer when passed to a function, and why you always pass a length as well. Print an address, a value and sizeof for the same array.
- Hour 2: strings and structs. A string is a char array ending in a NUL byte; strlen, snprintf and why strcpy is dangerous. A struct groups fields; print sizeof of your reading struct and see whether it equals the sum of its fields (padding).
- Different from languages you know: nothing checks array bounds, a string does not know its own length, and writing past the end of a buffer is silent corruption until the sanitiser catches it.

Compiler stages (1 h)

Run a small file through each stage by hand: cc -E (preprocess), cc -S (to assembly; read a few lines), cc -c (to an object file), then link. On a Mac cc is Apple's clang; the stages are the same for GCC, and arm-none-eabi-gcc arrives in the Firmware Lab.

Math hour: rate of change and the idea of a limit

1. Temperature rises from 21.0 °C to 23.4 °C over 5 minutes. Average rate of change?
2. For $f(x) = x^2$, compute the slope between $x = 2$ and $x = 2 + h$ for $h = 1$, 0.1 and 0.01. What value does it approach?
3. One ADC count of jitter near 25 °C is about 0.09 °C. At a 0.5 s sample interval, how big is the apparent rate of change from one count of noise between neighbouring samples?
4. In Python, compute the rate of change of your logged temperature between neighbouring rows and plot it. What do you see, and why?

Answers: (1) 0.48 °C/min. (2) 5, 4.1, 4.01, approaching 4. (3) 0.09 / 0.5 = 0.18 °C/s, or about 10.8 °C/min, from noise alone. (4) A very spiky line: differencing amplifies noise, which is why the later weeks average before differentiating.

Cold question

What port name does your Mac use, and how do you find it? What does a pointer hold?

Answer key: something like /dev/cu.usbmodem1101, found by comparing ls /dev/cu.* with and without the board plugged in, or from Tools > Port. A pointer holds a memory address; its type says what kind of value lives at that address.

Done when

- ☐ Explain UART, I2C and SPI in a sentence each
- ☐ Python reads the Arduino over pyserial
- ☐ Timestamped readings append to a CSV file
- ☐ Plot the CSV with matplotlib
- ☐ Push code and a README to GitHub
- ☐ C: pointers, arrays, strings and structs; compile a small C program with a Makefile

File this week

The repo link, the README (what it does, wiring photo, how to run, the plot), notes on every way the reader broke and how you fixed it, 3 more tagged postings, and the journal entry.

Week 5 (23 Nov): Soldering

Goal: By Sunday your week 3 sensor circuit is soldered onto perfboard with strain-relieved leads, every joint checked, and it logs the same data as the breadboard version.

Covers: soldering/rework, memory management, multimeter.

Needs: the kit above, your week 3 parts, scrap perfboard and components.

Sessions (20 h)

- S1, 3.5 h: Learn soldering fundamentals and safety (1.5 h) and tools and materials (0.5 h). Lab 5.1 first half (1.5 h).
- S2, 3.5 h: Learn joint inspection (0.5 h). Lab 5.1 second half (0.5 h). Lab 5.2 (1 h). Learn wiring and strain relief (0.5 h). Lab 5.3 (1 h).
- S3, 3.5 h: Learn perfboard layout (0.5 h). Layout drawing (1 h). C thread hour 1 (1 h). Dry-fit parts (1 h).
- S4, 3.5 h: Lab 5.4 assembly (2 h). Learn continuity checking (0.5 h). C thread hour 2 (1 h).
- S5, 3.25 h: Lab 5.5 (1 h). Math hour (1 h). Photo log, notes and 3 tagged postings (1.25 h).
- S6, 2.75 h: Cold question and journal (0.75 h). Spare (2 h).

Key ideas

- Safety: glasses on (clipped leads fly and solder spits), ventilation for the flux fumes, the iron always back in its stand, wash your hands after handling solder, and never touch a tip you have not checked.

- Keep the tip clean and tinned. Wipe it on brass wool rather than a wet sponge, which shocks the tip.
- Technique: touch the iron to the pad and the lead together for 1-2 s, feed the solder onto that junction (not onto the iron), take the solder away, then the iron, and do not move anything until the joint sets.
- A good joint is shiny or satin, with a smooth concave fillet that has wetted both the pad and the lead. Cold joints look dull, grainy or cracked, or sit as a ball on top. Also look for bridges between pads, too little solder and too much.
- Desoldering: wick works by pressing braid onto the joint with the iron; a pump sucks the molten solder away. Adding a little fresh solder first helps heat transfer.
- Strain relief: feed a wire through the board hole before soldering so the wire is held mechanically, not only by the solder. Add heat-shrink, and use stranded wire for anything that flexes.
- Perfboard: if yours is isolated pads, you join them with component leads, bare wire or solder bridges; on stripboard, plan where to cut the strips. Keep the thermistor away from heat sources on the board and leave a row of margin around the edge.

Lab 5.1: Practice joints (2 h)

On scrap perfboard make 20 through-hole joints with resistors and 10 with wire to pad. Inspect each under bright light, ideally with a magnifier, and classify it: good, cold, too little, too much or bridged. Photograph your best and worst. Aim for the last 10 joints all good.

Lab 5.2: Desolder (1 h)

Remove five scrap components: three with wick and two with a pump. Do not lift any pads. Resolder one and compare the joints.

Lab 5.3: Strain-relieved leads (1 h)

Make three leads of 100-150 mm: two wires for the thermistor, two for the LDR, and a four-way set for the 5 V, GND, A0 and A1 header connection. Thread each wire through the board, solder it, add heat-shrink, and gently tug-test: the joint must not move. Check continuity along each lead.

Lab 5.4: Perfboard assembly (3 h)

Build the week 3 sensor circuit: the thermistor divider and the LDR divider, with a 4-pin header (5 V, GND, A0, A1) that you connect to the Arduino with jumper wires. The thermistor and LDR sit on their leads so they can poke out of an enclosure in week 6.

First draw the layout on a grid: part positions, which connections are lead-to-lead, which need wire, and where the header goes. Dry-fit, then solder in order of height, shortest parts first. Trim leads after soldering, not before.

Lab 5.5: Continuity and power-up (1 h)

With everything unpowered:

1. Check continuity along every net you drew, joint by joint.
2. Check there is no continuity between neighbouring pads that should be separate.
3. Measure resistance between 5 V and GND: it should be in the kilohm range (roughly 5-15 kΩ with both dividers), never near zero.

Then power it from USB and run your week 4 logger for 10 minutes. Compare with the breadboard version at the same temperature: readings should agree to within a couple of °C. If not, use the meter to find which joint or net is wrong before touching the iron.

C thread (2 h)

- Hour 1: where things live. A program's code, initialised globals (.data), zeroed globals (.bss), the stack and the heap. Print the address of a local variable, a global, a static, a function and a malloc'd block (use %p) and see how they group. Write a recursive function with a missing base case and watch the stack overflow. Then return a pointer to a local variable, and read what -Wall says.
- Hour 2: the heap. malloc and free, leaks, use-after-free, fragmentation. Write a program with one deliberate bug of each kind and run it under -fsanitize=address; for leaks on macOS use leaks --atExit -- ./prog. Then explain why embedded code avoids malloc: fragmentation over months of uptime, unpredictable timing, a failure case to handle, and very little RAM.
- On the Uno: the ATmega328P has 2 KB of SRAM. After compiling a sketch, read the Global variables line in the IDE: that is your static RAM use out of 2048 bytes. Check that your week 3 sketch's number makes sense, and that strings in Serial.println(F(...)) are held in flash instead of RAM.

Math hour: limits and continuity

1. Evaluate the limit of $(x^2 - 4) / (x - 2)$ as x approaches 2.
2. Your divider gives count = $1023 \times R / (10\,k\Omega + R)$. What does the count approach as R becomes very large, and as R approaches 0? What does that say about adc_to_celsius()'s fault check?
3. Is a clamp that holds a value to the range 0 to 1023 continuous? Is a step from 0 to 1 at t = 0 continuous, and what are its one-sided limits?

4. An ADC's output is a staircase. Where is it discontinuous, and does that matter at the scale of 4.9 mV per count?

Answers: (1) 4, by factoring to x + 2. (2) 1023 as R grows (open sensor), 0 as R approaches 0 (shorted sensor), which are exactly the two values the fault check rejects. (3) The clamp is continuous (its pieces join); the step is not: the limit from the left is 0, from the right is 1, so the limit does not exist. (4) At every step edge, but the jumps are tiny compared with the signal, so treat it as continuous with quantisation error.

Cold question

Why heat the pad rather than the solder? What lives on the stack and what on the heap?

Answer key: solder flows toward hot metal. Heating the pad and lead together lets the flux clean both surfaces and the solder wet them; solder melted on the iron and dabbed onto cold metal makes a cold joint. The stack holds local variables and return addresses, created and released automatically as functions are called and return. The heap holds blocks you request with malloc and must release yourself.

Done when

- ☐ Make clean joints; spot a cold joint by sight
- ☐ Desolder a component with wick or a pump
- ☐ Add strain relief to every wire
- ☐ Move your week 3 circuit onto perfboard
- ☐ Check continuity on every joint before powering on
- ☐ C: stack vs heap, static allocation, why embedded code avoids malloc

File this week

The layout drawing, joint photos (best and worst), the RAM figure from the IDE, notes on any rework, 3 more tagged postings, and the journal entry.

Week 6 (30 Nov): Consolidation build and Gate 1

Goal: By Sunday you have a standalone two-sensor logger on perfboard in a printed enclosure, running a state machine with a ring buffer, documented on GitHub, and you have scored Gate 1.

Covers: state machines, algorithms and data structures, oscilloscope, breadboarding; tasters: CAN, USB.

Protected week: this week carries a gate, a taster and a decision, so do not schedule anything else heavy. Here, standalone means the soldered board and the Uno live together in one enclosure and run without the breadboard. It is still plugged into your Mac for power and CSV logging, because the plan has no SD card.

Sessions (20 h)

- S1, 3.5 h: Learn state machines and ring buffers (2.5 h). Lab 6.1 part 1 (1 h).
- S2, 3.5 h: Lab 6.1 part 2 (0.5 h). Lab 6.2 (2 h). Learn enclosure and print preparation (1 h); book or start the print.
- S3, 3.5 h: Lab 6.3 (1 h). Lab 6.4 integration (1 h). Learn oscilloscope basics (1 h). Tag every saved posting as firmware-heavy or field-and-systems-heavy (0.5 h).
- S4, 3.5 h: Lab 6.5 oscilloscope session (1.5 h). Lab 6.6 enclosure fit (1 h). Math hour (1 h).
- S5, 3.5 h: Tasters, 2 × 45 min (1.5 h). README, wiring diagram and photos (1.25 h). Gate 1 scoring (0.75 h).
- S6, 2.5 h: Finish Gate 1 (0.25 h). Cold question (0.25 h). Spare (2 h).

Key ideas

- A state machine is a set of states, the events that move between them, and the actions on each move. Draw it first: boxes for states, arrows labelled with the event that triggers them. Your week 2 ledState was the smallest possible one.
- Implement it as an enum and a switch in loop(), with no delay() calls, and make all state changes in one place so you can find them.
- A ring buffer is a fixed array with a head index (where the next item goes), a tail index (where the oldest item comes out) and a count. It decouples producing from consuming: sampling can carry on while slow serial output catches up. At 9600 baud a 30-byte line takes about 31 ms to send; at 115200, about 2.6 ms.
- Decide what happens when the buffer is full (drop the oldest or refuse the newest), write it down, and count the overruns.
- Static allocation only: the buffer is a fixed-size array. With 16 entries of a 12-byte reading, it costs 192 bytes of the Uno's 2048.

Lab 6.1: Ring buffer (1.5 h)

Write push, pop and count functions over a fixed array of 16 readings (a struct holding millis, thermistor count, temperature and LDR count). Keep them in plain C with no Arduino calls. Test them first on your Mac with a few assert statements in a small program built with your week 4 Makefile: fill it, overfill it, empty it, wrap around.

Lab 6.2: State machine firmware (2 h)

A suggested design, which you can change as long as you can justify it:

- IDLE: waiting for the 500 ms sample timer. The button toggles logging on and off. Slow LED blink.
- SAMPLING: read both channels (average a few readings), convert the temperature, push a reading into the ring buffer, move on.
- LOGGING: drain the ring buffer to Serial as CSV lines, then return to IDLE.
- FAULT: entered when the thermistor count is at 0 or 1023 for 3 samples in a row. Fast LED blink, a # FAULT line on Serial, retry every 2 s, and return to IDLE after 3 good readings.

Draw the diagram before writing code. Check the IDE's RAM figure afterwards.

Lab 6.3: Update and test the logger (1 h)

Run your week 4 Python reader against the new firmware for 15 minutes. Confirm the CSV format is unchanged, FAULT lines are ignored cleanly, and you can force a FAULT by unplugging the thermistor.

Lab 6.4: Integration (1 h)

Mount the Uno and the perfboard together, with the thermistor and LDR leads routed to where the enclosure openings will be. Re-run the continuity checks from week 5 after any rewiring.

Lab 6.5: Oscilloscope session (1.5 h)

At the makerspace or on a borrowed scope:

1. Set the probe to 10x, adjust compensation on the scope's calibration signal, and clip the ground lead to the Arduino's GND.
2. View a PWM pin with analogWrite(128): measure frequency and duty cycle with the scope's measurements and with cursors. Expect about 490 Hz and 50% on pin 9, about 980 Hz on pins 5 and 6, then repeat at analogWrite(64) and expect 25%.
3. View the thermistor divider node on DC coupling. Expect about 2.5 V at 25 °C. Warm the thermistor and watch the level move. Zoom in on the noise.
4. View the UART TX line at 9600 baud. A bit lasts about 104 µs (about 8.7 µs at 115200). Trigger on the falling edge and decode one character by eye. For example, 'A' is 0x41, sent least significant bit first: start bit 0, then 1 0 0 0 0 0 1 0, then stop bit 1.

Lab 6.6: Enclosure (1 h plus print time)

Measure your boards in mm. The Uno is 68.6 × 53.4 mm with four 3.2 mm mounting holes, so M3 standoffs fit. Allow at least 2 mm clearance each side. You need a USB cut-out, a window for the LDR, a slot or hole so the thermistor lead sits in open air away from board heat, and strain relief where the leads exit. Start from a simple box model (a generator or an existing design you modify) rather than learning CAD properly; Fusion 360 arrives in week 16. PLA is fine for a bench logger. Print, test-fit, and note what you would change.

Math hour: the derivative as a slope

1. For $f(x) = x^2$, use the limit of $((3 + h)^2 - 9) / h$ as h approaches 0 to find the slope at $x = 3$.
2. What is the derivative of $y = mx + b$, and what does that say about a calibration line?
3. From your week 3 predictions (788 counts at 0 °C, 512 at 25 °C, about 270 at 50 °C), compute the slope from 0 to 25 °C, from 25 to 50 °C, and from 0 to 50 °C.
4. In Python, compute a central-difference rate of change on your logged temperature and flag any value above 1 °C per second as a suspect spike.

Answers: (1) $(6h + h^2) / h = 6 + h$, so 6. (2) m, a constant slope: the sensitivity is the same everywhere, which a thermistor's is not. (3) -11.04, -9.68 and -10.36 counts per °C: the slope changes along the curve. (4) A central difference averages the neighbours on either side, so it is less noisy than a forward difference.

Landscape tasters (2 × 45 min)

Watch Microchip University's "Introduction to USB 2.0" and "CAN Bus: A Beginners Guide". For each, write one sentence on which jobs use it. For orientation: CAN is common in vehicles and in industrial, agricultural and marine machinery; USB shows up in firmware for anything that plugs into a computer.

Gate 1 (Go / Adjust / Stop)

Score each question 1-5 and write one sentence of evidence:

- Can you build, solder and log without a tutorial?
- Did debugging absorb you or drain you?
- Did C and bit-level detail energise you or drain you?

Then look at your journal: the energy pattern across weeks 1-6, your most absorbing and most draining tasks, and the hours you actually logged against the 20-hour plan. Re-tick the skills map for everything from weeks 1-6. Count your tagged postings (the target is 15, each marked firmware-heavy or field-and-systems-heavy) and name the two roles that now look most plausible.

Circle a decision. Go: carry on into Phase B. Adjust: write one sentence on what changes (hours, scope, pace). Stop: pause and revisit the role list before going further.

Cold question

What happens if you remove each component?

Answer key for your logger:

- Thermistor missing: A0 is pulled to 5 V through the fixed resistor, the count is 1023, and the firmware enters FAULT.
- Thermistor shorted: the count is 0, and the firmware enters FAULT.
- Fixed resistor missing: A0 has no path to 5 V and floats or drifts, so the temperature is garbage. FAULT may or may not catch it; that is a design weakness to note.
- LDR missing: A1 sits at 0, which looks exactly like darkness. A single divider cannot tell a fault from a legitimate extreme.
- LED resistor missing: the LED draws more than the pin's rating and fails, possibly taking the pin with it.
- Button missing: INPUT_PULLUP keeps the pin HIGH, so logging never toggles, which is a safe default.
- USB cable removed: no power, and no data.

Done when

- ☐ Standalone two-sensor logger on perfboard, logging to CSV
- ☐ Rewrite the logger firmware as a state machine (idle, sampling, logging, fault) with a ring buffer of readings
- ☐ 3D-printed enclosure made at the makerspace
- ☐ Oscilloscope session: view the PWM signal and the thermistor divider; measure frequency and duty cycle
- ☐ README, wiring diagram and photos on GitHub
- ☐ Landscape tasters (2 × 45 min): CAN bus and USB; one sentence on which jobs use each
- ☐ Gate 1 completed

File this week

The state diagram, the ring buffer test program, scope screenshots or photos, the enclosure files and photos, the README and wiring diagram on GitHub, your Gate 1 sheet, and the journal entry.